

```
from google.colab import files
```

```
uploaded = files.upload()
```

Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser

session. Please rerun this cell to enable.

Saving StudentsPerformance.csv to StudentsPerformance.csv

```
import pandas as pd
```

```
df = pd.read_csv("StudentsPerformance.csv")
```

```
df.head()
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75

```
display(df)
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75
...	...	...	...	...	...	...	...	...
995	female	group E	master's degree	standard	completed	88	99	95
996	male	group C	high school	free/reduced	none	62	55	55
997	female	group C	high school	free/reduced	completed	59	71	65
998	female	group D	some college	standard	completed	68	78	77
999	female	group D	some college	free/reduced	none	77	86	86

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 1000 entries, 0 to 999
```

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	gender	1000 non-null	object
1	race/ethnicity	1000 non-null	object
2	parental level of education	1000 non-null	object
3	lunch	1000 non-null	object
4	test preparation course	1000 non-null	object
5	math score	1000 non-null	int64
6	reading score	1000 non-null	int64
7	writing score	1000 non-null	int64

dtypes: int64(3), object(5)

memory usage: 62.6+ KB

```
df.isnull().sum()
```

	0
gender	0
race/ethnicity	0
parental level of education	0
lunch	0
test preparation course	0
math score	0
reading score	0
writing score	0

dtype: int64

```
df["average_score"] = (
    df["math score"] +
    df["reading score"] +
    df["writing score"]
) / 3

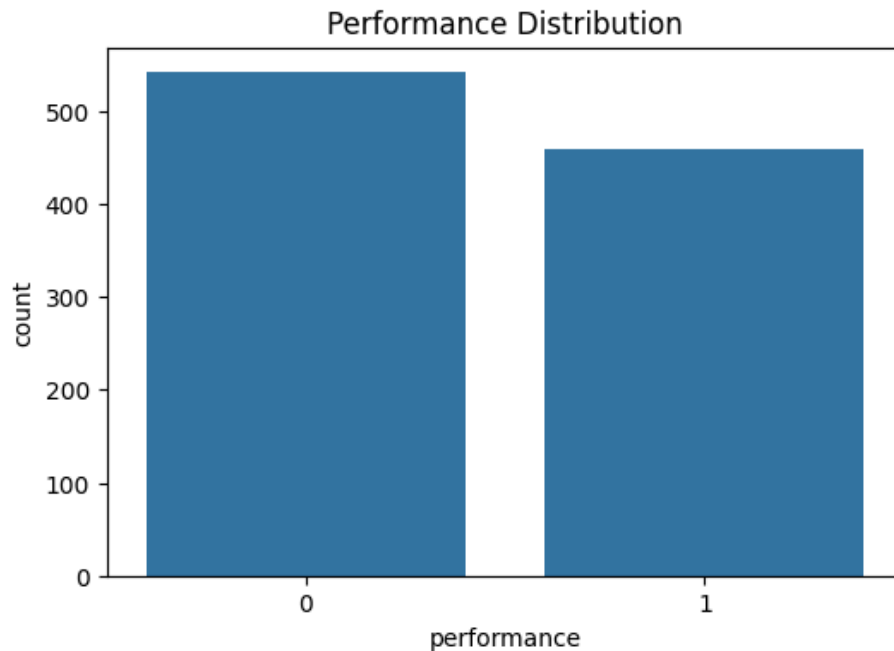
df["performance"] = df["average_score"].apply(
    lambda x: 1 if x >= 70 else 0
)

df[["average_score", "performance"]].head()
```

	average_score	performance
0	72.666667	1
1	82.333333	1
2	92.666667	1
3	49.333333	0
4	76.333333	1

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(6,4))
sns.countplot(x="performance", data=df)
plt.title("Performance Distribution")
plt.show()
```



```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

df.head()
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score	average
0	0		1	1	1	72	72	74	72
1	0		2	4	1	69	90	88	82
2	0		1	3	1	90	95	93	92
3	1		0	0	1	47	57	44	49
4	1		2	4	1	76	78	75	76

```
X = df.drop(["average_score", "performance"], axis=1)
y = df["performance"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (1000, 8)
y shape: (1000,)
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)
```

```
(800, 8)
(200, 8)
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

rf = RandomForestClassifier(random_state=42)

rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

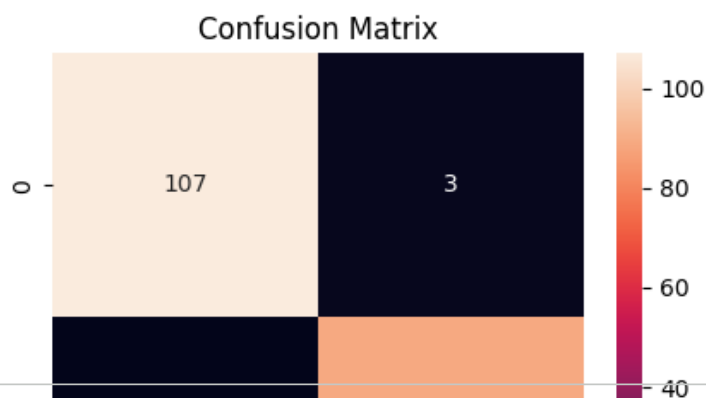
print("Accuracy:", accuracy)
```

```
Accuracy: 0.98
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.show()
```



```
import joblib
```

```
joblib.dump(rf, "student_performance_model.pkl")
```

```
print("Model Saved Successfully")
```

Model Saved Successfully

```
import joblib
```

```
# Load the model
```

```
loaded_model = joblib.load("student_performance_model.pkl")
```

```
print("Model loaded successfully:", loaded_model)
```

Model loaded successfully: RandomForestClassifier(random_state=42)

```
import joblib
```

```
# Load the model
```

```
loaded_model_2 = joblib.load("student_performance_model.pkl")
```

```
print("Model loaded successfully:", loaded_model_2)
```

Model loaded successfully: RandomForestClassifier(random_state=42)

```
import matplotlib.pyplot as plt
```

```
models = ['Random Forest']
```

```
accuracy = [0.98]
```

```
plt.bar(models, accuracy)
```

```
plt.title("Model Accuracy Comparison")
```

```
plt.ylabel("Accuracy")
```

```
plt.show()
```

